RAG Studio

Product Requirements Document

Design, Build, Evaluate and Deploy Production-Grade RAG Systems

Version 1.0 | June 2026

Author: Satyam Pandey

Table of Contents

1. What is RAG Studio?

RAG Studio is an open-source developer tool that makes it straightforward to build, test, and run AI systems that can answer questions about your own documents. Instead of relying on a general-purpose AI that guesses from its training data, RAG Studio connects an AI model to files you actually own, so every answer it gives is grounded in real content you can check.

Think of it as a workbench. You load up your documents, pick your settings, run a few experiments, and end up with a working question-answering system you can trust. Everything the AI does is visible, so you are never left wondering why it gave a particular answer.

1.1 Document Metadata

Field	Details
Project Name	RAG Studio
Version	1.0
Author	Satyam Pandey
GitHub	https://github.com/SatyamPandey07/RAG-Studio
Status	Active Development
Date	June 2026
Tech Stack	FastAPI (Python), React + TypeScript (Vite), Qdrant, SQLite, Docker

2. The Problem We Are Solving

Ask any standard AI chatbot about a document sitting on your hard drive and it will either admit it does not know, or worse, make something up. This is a real problem for teams that want to use AI to help with internal knowledge bases, support documents, research papers, or any proprietary data.

The technical community has a name for the made-up answer problem: hallucination. RAG (Retrieval-Augmented Generation) is the most widely adopted solution. The idea is simple: before the AI writes an answer, it looks up the most relevant passages from your documents and uses those as context. This way it is working from facts rather than guessing.

But setting up a good RAG system is harder than it sounds:

- Splitting documents into the right-sized pieces matters a lot. Too big and retrieval is noisy. Too small and you lose context.
- Balancing keyword search and meaning-based search is dataset-specific. What works for legal contracts may not work for engineering specs.
- Measuring quality requires proper tooling. Eyeballing a few answers does not tell you how the system performs across hundreds of queries.
- Most RAG tools are black boxes. You send a question, get an answer, and have no idea what happened in between.

RAG Studio addresses all of these. It gives you full visibility into every step, lets you experiment with different settings, and provides real numbers to back up your configuration choices.

3. Goals for This Version

3.1 What We Are Building

- A zero-configuration demo mode so any developer can be up and running in under five minutes, with no API keys and no cloud setup required.
- A set of interactive studios (Chunking, Embedding, Search, Re-ranking, Evaluation) where each part of the pipeline can be tuned and compared side by side.
- Support for multiple AI providers (Google Gemini, OpenAI, Cohere) switchable through a single configuration file.
- Quantitative evaluation scores (Recall, Faithfulness, Groundedness) so there is a concrete way to measure whether changes actually help.
- A clean deployment path using Docker Compose, making it straightforward to run on a local machine or a cloud server.
- Auto-generated API documentation so other systems can integrate with RAG Studio as a backend service.

3.2 What Is Out of Scope (Version 1.0)

- Multi-tenant hosting with separate accounts and billing (planned for a later version).
- Fine-tuning or retraining the underlying AI models.
- Real-time streaming chat responses (on the roadmap for version 1.1).
- Mobile applications for iOS or Android.
- Support for image, audio, or video files as document inputs.

4. Who Is This For?

Primary - AI and ML Engineers

These are people who build RAG pipelines professionally. They want fine-grained control over chunking, retrieval, and re-ranking, and they need numbers to justify their configuration choices to the rest of the team. RAG Studio gives them that control and those numbers in one place.

Secondary - Data Scientists

Researchers and analysts who want to experiment with document question-answering on private datasets. They care about reproducibility, being able to run the same experiment twice and get the same result, and want to export their evaluation results for reporting.

Also useful for - Developers and Tech Leads

Developers evaluating whether to adopt RAG for a product they are building. They need a quick demo to see what is possible without setting up infrastructure, and they want to understand the API surface area before committing to an integration.

Future - Non-Technical Users

Product managers and domain experts who want to upload documents and ask questions without knowing anything about vectors or chunking. A simplified mode for this audience is planned for a future release.

5. How the System Works

5.1 Overview

RAG Studio is split into two main parts that talk to each other. The frontend is what you see in your browser - a visual interface with all the studios, the chat window, and the dashboards. The backend runs the actual AI pipeline: it receives your documents, breaks them into pieces, stores the meaning of each piece as numbers in a special database, and retrieves the right pieces when you ask a question.

When you ask a question, the system finds the most relevant passages from your documents, combines them with your question, sends the package to an AI model (like Google Gemini), and returns the answer along with the source passages it used.

5.2 Technology at a Glance

Layer	Technology	What it does
Backend	Python 3.10 / FastAPI / Uvicorn	Handles all the behind-the-scenes logic, API calls, and RAG pipeline steps
Frontend	React 18 / TypeScript / Vite	The visual interface you interact with in your browser
Vector Database	Qdrant	Stores the meaning-based representations of your documents for fast search
Regular Database	SQLite (demo) / PostgreSQL (production)	Keeps track of documents, results, logs, and user feedback
AI Providers	Google Gemini 2.0, OpenAI, Cohere	The AI models that read, understand, and answer questions about your documents
Deployment	Docker / Docker Compose	Packages everything so the app runs the same way on any machine

5.3 Two Ways to Run It

Demo Mode (the default)

When you first start RAG Studio, it runs in demo mode. Everything works offline using sample data, a lightweight local database, and a built-in search engine. No API keys, no cloud accounts, no setup beyond installing the app. This mode is perfect for exploring the features, running tests, or showing colleagues what the system can do.

Live Mode

Once you are ready to connect real AI models to your own documents, you switch live mode on by adding your API key to a single config file. RAG Studio then generates real embeddings and routes

questions through the AI provider of your choice (Gemini, OpenAI, or Cohere). You can switch between providers by changing one line.

6. Features and Modules

RAG Studio has eight modules. Each one focuses on a different part of the pipeline, and each one can be used independently or in sequence. The table below gives a quick overview before the detailed descriptions.

Module	What it lets you do	You know it works when...
Knowledge Sources	Upload your own files (PDFs, Word docs, spreadsheets) so the AI can read them	Files are indexed and ready to query within 30 seconds
Chunking Studio	See how the app splits documents into pieces and tweak the settings	Side-by-side view updates in under 1 second as you move the slider
Embedding Studio	Peek under the hood at how the AI mathematically represents your text	You can inspect the vector for any chunk on demand
Hybrid Search	Mix keyword search and meaning-based search to get the best of both	Changing the slider updates results in under 2 seconds
Re-ranking	Let a second AI model double-check and reorder the search results for better accuracy	The reordered list is different from the original, and latency is shown
Chat Playground	Ask questions and see exactly which parts of your documents the AI used to answer	Every answer shows source chunks and cost breakdown
Evaluation Studio	Run tests to measure how accurate and trustworthy your pipeline is	Scores are shown per question and as a summary you can export
Monitoring	Track usage, costs, and whether users found the answers useful	Stats update within 5 seconds of each query

6.1 Knowledge Sources

This is where you load up the documents you want the AI to know about. You can upload files via a drag-and-drop interface, and the system will parse, clean, and index them automatically.

- Accepted file types: PDF, TXT, Markdown, Word documents, CSV, JSON, HTML.
- A status indicator shows you where each file is in the process: queued, parsing, chunking, embedding, indexed, or error.
- You can delete individual files and reindex without starting over.
- All document metadata (file name, size, number of chunks, timestamp) is stored so you can audit what is in your knowledge base at any time.

6.2 Chunking Studio

Before a document can be searched, it needs to be cut into smaller pieces. The Chunking Studio lets you see exactly how that happens and experiment with different approaches before committing to one.

- Supported strategies: Recursive (splits on natural boundaries like paragraphs), Fixed Size (equal-length chunks), Semantic (respects sentence boundaries), Token-based (splits by AI token count), and Markdown-aware (splits on headers).
- Sliders let you adjust chunk size and the amount of overlap between chunks.
- A side-by-side view shows how two different strategies handle the same document so you can compare before deciding.
- Chunk count and average length update in real time as you move the sliders.

6.3 Embedding Studio

Once a document is chunked, each piece gets turned into a list of numbers (a vector) that captures its meaning. The Embedding Studio makes this process visible.

- Select any chunk from your knowledge base and inspect its vector - the raw numbers the AI uses internally.
- Compare embeddings from different AI providers to understand how they represent meaning differently.
- A future update will include a 2D visual map showing how your chunks cluster together, making it easy to spot gaps in your knowledge base.

6.4 Hybrid Search and Fusion

When you ask a question, RAG Studio actually runs two searches at the same time: one based on meaning (semantic search) and one based on keywords (like a traditional search engine). The Hybrid Search module lets you control how much weight each one gets.

- Dense retrieval uses Qdrant to find chunks whose meaning is closest to your question.
- Sparse retrieval uses BM25 to find chunks with matching keywords.
- A fusion algorithm (Reciprocal Rank Fusion) merges both results into a single ranked list.
- An interactive slider (0 = pure keyword, 100 = pure semantic) shows you the results change in real time, so you can tune it to your specific dataset.

6.5 Re-ranking

After the initial search, a second AI model (a cross-encoder) takes a closer look at the top results and reorders them for higher accuracy. This extra step is slower but often catches cases where the first search got the ranking slightly wrong.

- Supports Cohere Rerank and pluggable custom cross-encoders.
- A before-and-after view shows exactly which chunks moved up or down the list after re-ranking.
- The extra time re-ranking takes is shown alongside the quality improvement, so you can make an informed trade-off.

6.6 Chat Playground

The Chat Playground is where you actually talk to your documents. It looks and feels like a standard chat interface, but with one important difference: for every answer, you can expand a panel that shows exactly which chunks the AI read, how confident it was in each one, and what the whole query cost in time and money.

- Multi-turn conversation with context memory within a session.
- An expandable 'Explain Retrieval Steps' section shows retrieved chunks, similarity scores, re-rank scores, latency, and estimated cost for every response.
- A system prompt editor lets you customise how the AI responds (formal vs. casual, short vs. detailed, etc.).
- A model selector lets you switch AI providers without touching any code.
- Thumbs up / thumbs down buttons on each answer feed into the monitoring dashboard.

6.7 Evaluation Studio

The Evaluation Studio is for when you want proper numbers rather than just a feel for how well things are working. You define a test set of questions with known correct answers, run the pipeline against them, and get scores back.

- Upload or manually define a test set with questions, expected answers, and optionally the document chunks those answers should come from.
- The system computes Recall@K (did the right chunks get retrieved?), Faithfulness (is the answer supported by the context?), and Groundedness (can every claim be traced to a source?).
- Results break down per question so you can see exactly where the pipeline struggles.
- Export the full report as JSON or CSV for sharing or further analysis.

6.8 Monitoring Dashboard

Once the system is running, the Monitoring Dashboard gives you a live view of what is happening: how many queries are coming in, what they are costing, and whether users are finding the answers useful.

- Query volume over time, broken down by hour, day, or week.
- Cost breakdown separating LLM token costs from embedding API costs.
- User feedback summary: thumbs-up rate, thumbs-down rate, and unanswered queries.
- The most-asked questions and the most-retrieved document chunks, helping you spot gaps in your knowledge base.

7. Performance and Reliability

These are the targets the system should hit. They are not aspirational - they are the baseline for a release to be considered stable.

Area	Requirement	Target
Speed	Query round-trip time (live mode)	Under 5 seconds at the 95th percentile
Speed	Starting up in demo mode	Under 10 seconds, no internet required
Scalability	Documents per knowledge base	1,000+ files, 1 million+ chunks
Reliability	Uptime (self-hosted Docker)	99.5% target
Security	API key handling	Keys stay on the server in .env files, never sent to the browser
Portability	Supported platforms	Linux, macOS, Windows via Docker Compose

8. How to Set It Up

Getting RAG Studio running on your machine takes about five minutes. Here is the shortest path:

What you need first

- Node.js version 18 or newer
- Python version 3.10 or newer
- Docker and Docker Compose (if you want the one-command setup)

Option A - Docker Compose (recommended)

From the root of the repository, run one command: docker compose up. That starts the backend, the frontend, and the Qdrant database together. Open your browser at http://localhost:3002 and you are in.

Option B - Running each part manually

Open two terminal windows. In the first one, navigate to the backend folder, create a Python virtual environment, install the dependencies, and start the server. In the second terminal, navigate to the frontend folder, install the packages, and start the dev server. Full step-by-step commands are in the repository README.

Switching to Live Mode

Copy the file called .env.example in the backend folder and rename it .env. Change DEMO_MODE from true to false, add your Google Gemini API key (free to get at aistudio.google.com), and optionally add keys for OpenAI or Cohere if you want those providers too. Restart the backend and you are live.

9. Roadmap

Development is split into four phases. Each phase builds on the one before it. Timelines are estimates and may shift based on contributor availability and feedback from early users.

Phase	When	What gets shipped
Phase 1 - Foundation	Q3 2026	Knowledge Sources, Chunking Studio, Embedding Studio, basic Chat Playground in demo mode
Phase 2 - Core AI	Q4 2026	Hybrid Search, Re-ranking, live mode with Gemini / OpenAI / Cohere integration
Phase 3 - Quality	Q1 2027	Evaluation Studio with Recall, Faithfulness and Groundedness scoring
Phase 4 - Operations	Q2 2027	Monitoring dashboard, multi-user support, cloud deployment guides for AWS / GCP / Azure

Progress on each phase is tracked publicly on the GitHub repository via Issues and Project boards. Anyone is welcome to contribute.

10. Risks and How We Handle Them

Every project has things that could go wrong. Here are the ones worth keeping an eye on and what the plan is if they do.

Risk	Severity	How we handle it
Third-party AI APIs change or shut down	High	Providers are swappable via a single config line; demo mode always works offline
Qdrant releases a breaking change	Medium	Pin version in Docker Compose; run integration tests before upgrading
Very large files cause memory spikes	Medium	Stream uploads; enforce a 50 MB client-side limit with a clear error message
API keys accidentally exposed in the UI	High	Keys are stored server-side only in .env; the browser never sees them
AI judge gives biased evaluation scores	Low	Document the methodology; let users customise the judge prompt; support multiple judges

11. How We Measure Success

Success means different things at different stages. Here is what we are tracking:

Community Adoption

- 500 or more GitHub stars within six months of public launch.
- 100 or more repository forks, which indicates active customisation by other developers.
- 1,000 or more Docker image pulls per month.

Developer Experience

- A brand new user should go from git clone to asking their first question in demo mode in under five minutes.
- The automated test suite should pass on all supported platforms at a rate of 95% or higher.
- API endpoints that do not involve AI calls should respond in under 500 milliseconds.

Answer Quality

- A Faithfulness score above 0.85 on the standard benchmark dataset using the default Gemini 2.0 setup.
- Zero critical security vulnerabilities in any published release.

12. Glossary

If any term in this document was unfamiliar, this table should help.

Term	Plain-English meaning
RAG	Retrieval-Augmented Generation. A technique where an AI looks up relevant parts of your documents before writing an answer, rather than relying purely on its training data.
Chunking	Cutting a long document into smaller, manageable pieces so the AI can search them individually.
Embedding	A list of numbers that captures the meaning of a piece of text so similar ideas end up mathematically close together.
BM25	A classic keyword-matching algorithm - the same idea that powers traditional search engines.
RRF	Reciprocal Rank Fusion. A way of combining two ranked lists (keyword and semantic) into one sensible list.
Hallucination	When an AI makes something up that sounds plausible but is not actually true or supported by your documents.
Faithfulness	A score that checks whether every claim in an answer is supported by the retrieved text.
Groundedness	Similar to faithfulness - checks that the answer can be traced back to a specific source in your documents.
Qdrant	An open-source database built specifically for storing and searching embedding vectors.
Demo Mode	A no-setup mode that runs entirely on your machine with sample data and no API keys needed.

13. References

The following links are useful if you want to dig deeper into any part of the stack:

GitHub Repository: https://github.com/SatyamPandey07/RAG-Studio
FastAPI Documentation: https://fastapi.tiangolo.com
Qdrant Documentation: https://qdrant.tech/documentation
Google AI Studio (Gemini API keys): https://aistudio.google.com

End of Document